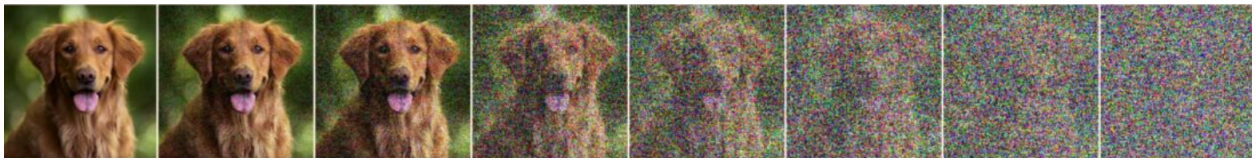
Diffusion Models



written by
Rudraksh Gupta
Raunak Pal

Index

A. From discriminative to generative models 3
B. Pre-Diffusion Landscape 4
C. Introduction of diffusion models 4
D . Physical Intuition: Non-Equilibrium Thermodynamics and Reversibility 6
E. Basic Structure of DDPM Models 6
1 . The Forward Diffusion Process: Entropic Decay 11
1.1 The Goal and Conceptual Mechanism 11
1.2 Components of the Forward Process 12
1.2.1 Forward diffusion kernel (FDK), 12
1.2.2 Variance schedule 12
a. The Formation of Global Structure (High t) 13
b. Ensuring Prior Convergence 13
c. Transition to Fine Details (Low t) 13
1.2.3 The Reparameterization Trick and Closed-Form Sampling 14
a. The Gaussian Reparameterization Trick 14
b. Derivation of Closed-Form Sampling 14
c. Impacts and Computational Significance 15
2. The Reverse Diffusion Process: Creating Order from Chaos 16
2.1 The Score Function 17
2.2 Connection to Diffusion 18
2.3 Langevin Dynamics 18
2.4 Connection to Score Matching 18
2.5 Annealed Langevin Dynamics (The "Annealing" Factor) 19
3. Variational Inference and the Optimization Objective 19
3.1 Introduction: The Intractability of Maximum Likelihood 19
3.2 Deriving the Variational Lower Bound 20
3.3 Decomposing the Objective 20
3.4 The Forward Posterior: The "Ground Truth" 21
3.5 The Final Simplified Loss Function (Lsimple) 22
F. The Neural Architecture: A Time-Conditional U-Net 23
G. Experimental Evaluation and Generative Performance 23
H. Future Directions: The Evolution toward Latent Diffusion 25

I. Real-World Applications and Computational Tools..... 26
1. DALL-E 2 (OpenAI).....26
2. Midjourney..... 26
3. Drug Discovery and Protein Design (DiffDock / RFdiffusion).....26
4. AudioLDM and Riffusion..... 26
J. Implementation of a DDPM on the MNIST Dataset from scratch..... 27
Data Preprocessing..... 27
Forward Diffusion Process.....28
Reverse Diffusion Process and Neural Network Architecture..... 29
K . References.....31

A. From discriminative to generative models

The transformation in machine learning from discriminative to generative modeling has been a big change . It signifies a transition from the simpler goal of **curve fitting** to the significantly more complex objective of **distributional understanding** .While discriminative models were common in the initial deep learning era,largely due to their comparative ease of optimization,the change toward generative frameworks required overcoming considerable computational and theoretical hurdles.

Discriminative models learn the boundary *between* classes to classify data (e.g., Logistic Regression, SVMs), focusing on conditional probability $P(Y|X)$; they're great for classification but can't generate data. Generative models learn the *distribution* of the data itself to understand how data is generated, allowing them to create new samples (e.g., GANs, VAEs, Naive Bayes) thus are often more complex and data-hungry . Generative model learns joint probability $P(X,Y)$ (probability of input and label together) . Discriminative models can be simplified into a basic curve fitting task . They are trained on a large dataset with their corresponding labels so that they can predict the labels of new data.

On the other hand training and developing a generative model has always been harder due to

- **Higher Dimensionality & Complexity:** Finding a separating line between two gro ups of points is significantly simpler than modeling the entire data space for instance, every possible pixel combination within an image.
- **Training Instability:** Several models like General Adversarial networks (GANs), which consist of a competing generator and discriminator, are prone to instability. This frequently results in one network dominating the other, causing issues like vanishing gradients or "mode collapse."

- **Data Distribution Challenges:** The process of uncovering detailed and complex relationships within real-world data is demanding. It is computationally intensive, requires very large datasets and significant computing power, and is susceptible to errors.
- **Interpretability & Grounding:** Understanding *why* a generative model created something specific is hard due to high-dimensional latent spaces. They learn patterns, not necessarily real-world truth, which may lead to "hallucinations" .
- **Assumptions & Bias:** Generative models often make assumptions about data structure (e.g., Gaussian distributions in older models) that might not hold true , inducing biases in the models.

B. Pre-Diffusion Landscape

For a decade, the field of deep generative modeling was characterized by a division between two main types of models, each with distinct strengths and weaknesses.

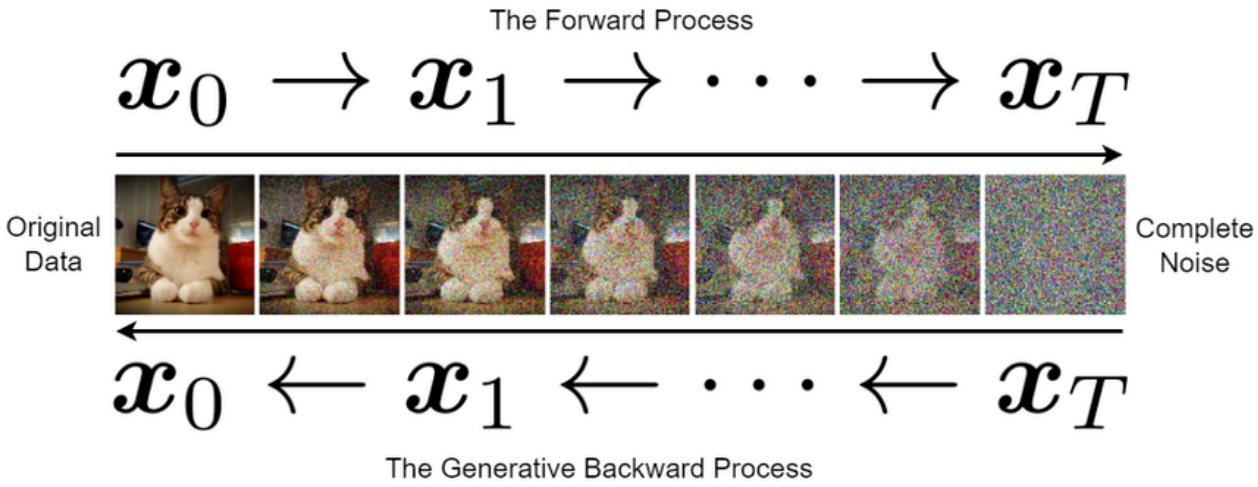
1. Likelihood-based models, such as **Variational Autoencoders (VAEs) and autoregressive models**, provide a strong probabilistic foundation with straightforward training objectives based on maximum likelihood estimation, crucially guaranteeing full coverage of the data distribution by avoiding the omission of any data modes. However, a major weakness is the tendency to produce perceptually inferior results; VAEs, limited by the Evidence Lower Bound (ELBO) and simple reconstruction losses, typically yield blurry samples lacking high-frequency details, while autoregressive models, despite their potential for high fidelity, are impractical for real-time applications due to computationally slow, sequential sampling that scales poorly with high-resolution data.

2. Implicit Generative Models, specifically **Generative Adversarial Networks** (GANs), conceptualize generation as a competitive, zero-sum game between a generator and a discriminator. While this mechanism has led to an unprecedented leap in the photorealism and visual quality of generated images, this visual superiority is achieved at a significant cost. Adversarial training is notoriously unstable and difficult to manage because the loss function does not reliably track the actual quality of the samples. Furthermore, a major weakness of GANs is their high vulnerability to "mode collapse," a failure where the model ignores the underlying diversity of the data distribution and instead focuses on reproducing only a small number of high-probability samples.

C. Introduction of diffusion models

Diffusion Models are generative models, meaning that they are used to generate data similar to

the data on which they are trained. Fundamentally, Diffusion Models work by destroying training data through the successive addition of Gaussian noise, and then learning to recover the data by *reversing* this noising process. After training, we can use the Diffusion Model to generate data by simply passing randomly sampled noise through the learned denoising process. Diffusion models are inspired by non-equilibrium thermodynamics. They define a Markov chain of diffusion steps to slowly add random noise to data and then learn to reverse the diffusion process to construct desired data samples from the noise.
Diffusion models are most prominently associated with image generation and other image processing tasks such as inpainting and super-resolution, but their applications extend to other domains including audio generation, drug design and molecule generation.



Diffusion models represent a significant advancement over previous generative techniques like Variational Autoencoders and Generative Adversarial Networks . Key differentiators include their approach to dimensionality and process: unlike VAEs, which compress data into a low-dimensional latent space in a single step, diffusion models maintain the data's dimensionality through a gradual, multi-step transformation. Furthermore, their objective function is a stable, non-adversarial one derived from the Evidence Lower Bound (ELBO), contrasting with the adversarial optimization used by GANs. This approach results in a model that establishes a mathematically rigorous framework for generation, achieving state-of-the-art Fréchet Inception Distance (FID) scores, which demonstrate superior diversity compared to GANs.
Diffusion models excel because they prioritize **perceptual realism** over statistical perfection unlike VAEs, which often produce blurry images to maximize "log likelihood", and GANs, which are notoriously unstable and prone to "mode collapse" , Diffusion models rely on a stable, iterative process. By functioning as a **"progressive decoder"**,similar to an artist sketching a rough outline before filling in fine details,they can focus on generating coherent high-level structures without getting confused by every single pixel at once. This allows them to accept a "worse" mathematical score for predicting invisible static noise (as the paper notes) in exchange

for generating images that look significantly sharper, more diverse, and more realistic to the human eye.

D . Physical Intuition: Non-Equilibrium Thermodynamics and Reversibility

The theoretical foundation of diffusion models draws a direct parallel to the dynamics of heat dissipation and molecular diffusion in thermodynamic systems. We may consider the **Forward Process** as a systematic increase in entropy. In this phase, a data distribution (analogous to a highly ordered state) is subjected to a stochastic process wherein Gaussian noise is incrementally injected over a finite time horizon, denoted as t=0 to t=T. This mirrors the physical phenomenon where a structured system degrades into thermodynamic equilibrium,a state of maximum entropy or isotropic noise,governed by the arrow of time.

The **Reverse Process**, therefore, postulates a violation of the second law of thermodynamics for the purpose of generative modeling. The objective is to model the time-reversal of this stochastic diffusion. By learning the conditional probability gradients (scores) at each temporal step, the neural network approximates the trajectory required to traverse from a high-entropy state (pure Gaussian noise) back to the low-entropy manifold of the data distribution. Essentially, the model learns to invert the diffusion equation, iteratively denoising the system to reconstruct complex structure from chaos.

E. Basic Structure of DDPM Models

At the heart of a DDPM are two opposing Markov chains. The first is a fixed **forward process**

that incrementally adds small amounts of Gaussian noise to an input image over a finite number of timesteps, T. This process gradually corrupts the data until, at timestep T, the image is indistinguishable from pure isotropic Gaussian noise. The second is a learned **reverse process** that is trained to undo the noising steps. Starting with pure noise, this process iteratively denoises the data to produce a coherent, clean image. Conceptually, this is represented by a forward Markov chain x0→x1→...→xT that degrades the data, and a learned reverse Markov chain x0←x1←...←xT that generates the data.

Notation / Variable	**Concept**	**Explanation**
x_0	**Original Data**	The clean, starting data point (e.g., the original, uncorrupted image) sampled from the unknown real data distribution $q(x_0)$.
x_t	**Noisy Data / Latent**	The data point at a specific step t in the diffusion process, which is a corrupted version of x_0. t ranges from 1 to T.
T	**Total Timesteps**	The total number of steps in the forward or reverse diffusion process (e.g., T=1000). This is the maximum time when the data is nearly pure noise.
$q(...)$	**Forward Process (Distribution)**	The fixed, known process of gradually adding noise to the data. It defines the "forward diffusion kernel" (FDK).
$p_\theta(...)$	**Reverse Process (Distribution)**	The learned generative process used to gradually remove noise and generate data samples. The subscript θ indicates that this process is parameterized

		(learned) by a neural network.
$q(x_t \| x_{t-1})$	**Forward Transition**	The conditional probability distribution defining how x_{t-1} becomes the slightly noisier x_t in one small step. This is a fixed Gaussian function.
$p_\theta(x_{t-1} \| x_t)$	**Reverse Transition**	The conditional probability distribution defines how the model estimates the previous, less noisy step x_{t-1} from the current noisy step x_t. This is the part the model learns.
β	**Variance Schedule / Diffusion Rate**	A set of fixed constants that controls the small amount of noise added at each time step t in the forward process. These values are small and often increase linearly.
α	**Signal Retention Factor**	Defined as $\alpha = 1 - \beta$. It represents the factor by which the previous signal is preserved (scaled down) at time t.
$\overline{\alpha}$	**Cumulative Signal Factor**	This is the total cumulative product of the α values from t=1 up to T. It determines how much of the original data x_0 signal remains at time t.

ε	**Standard Gaussian Noise**	A vector of pure, random noise sampled from the Standard Normal distribution $N(0, I)$. This is the noise added in the forward process and the noise the model ε_θ learns to predict.
$\varepsilon_\theta(x_t, t)$	**Noise Prediction Model**	A neural network function (often a U-Net) parameterized by θ. It takes a noisy sample x_t and the time step t as input and outputs an *estimate* of the noise $\varepsilon$$ that was added to create x_t from x_0.
$N(\mu, \sigma)$	**Normal (Gaussian) Distribution**	Represents the Gaussian probability distribution with mean μ and covariance matrix σ. In diffusion models, σ is usually $\beta\ I$ (identity matrix), leading to an isotropic Gaussian distribution.
I	**Identity Matrix**	A matrix that defines the covariance for isotropic (uniform in all directions) Gaussian noise/variance. When used in $N(..., \beta_t I)$, it means the noise is added equally to all dimensions.
μ_θ	**Learned Mean**	The predicted mean of the Gaussian distribution for the reverse transition $p_\theta(x_{t-1}\|x_t)$. This is the primary part the model is trained to estimate.

μ_t	**True Posterior Mean**	The analytical (known) mean of the reverse diffusion kernel, conditional on both the current state x_t and the original data x_0. The learned mean μ_θ aims to predict this quantity.
σ_θ	**Learned Variance**	The predicted covariance matrix of the Gaussian distribution for the reverse transition. In many DDPM implementations, this is fixed rather than learned.
$\nabla_x log(p(x))$	**Score Function**	The gradient of the logarithm of the probability density function $p(x)$. It points in the direction of steepest increase in probability density. Diffusion models show equivalence to learning this function.
$\|\ldots\|$	**Squared $\|\|$ Norm**	Represents the squared distance (e.g., mean squared error) between two vectors, commonly used as the final simplified loss function L_{simple} for training the noise prediction network.
L_{VLB}	**Variational Lower Bound (Loss)**	The fundamental objective function used to train the Diffusion model, based on approximating the negative log-likelihood of the data.
L_{simple}	**Simplified Loss**	An empirically found version of the training objective that simplifies to the Mean Squared Error (MSE) between the

		predicted noise ε_θ and the true noise ε.

1 . The Forward Diffusion Process: Entropic Decay

1.1 The Goal and Conceptual Mechanism

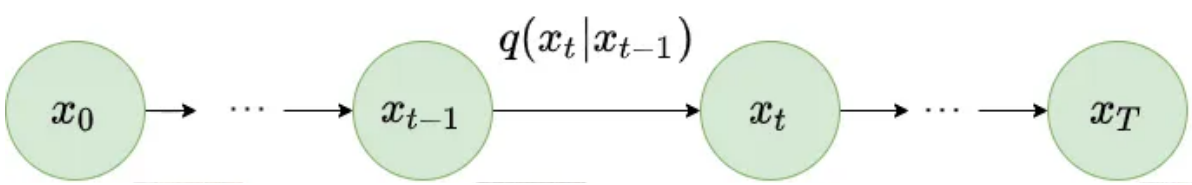
The purpose of the forward process is to transform the starting probability distribution of the data, q(x,0), which is unknown and highly complex, into a final distribution, p(x,T), that is simple and easy to understand: the **Standard Normal (Isotropic Gaussian) distribution** (N(0,I)).

In essence, since we cannot easily sample a real image directly, we instead learn how to reverse the process of turning an image into pure, simple noise (x_T), which we *can* easily sample.

The transformation happens through a sequence of steps defined as a **Markov chain**. A Markov chain is a sequence where the state at any time (x_t) depends only on the state immediately preceding it (x_{t-1}), and not on any steps before that

The forward diffusion mechanism is defined as a **stochastic process** because the transition from state x_{t-1} to x_t is not a deterministic function, but rather a sample drawn from a conditional probability distribution q($x_t | x_{t-1}$). This introduces random uncertainty at every timestep, ensuring that the trajectory from data to noise is unique and non-repeatable for every realization.

If you start with an original image, x0, the chain progresses through T steps, creating a sequence of increasingly noisy versions, known as **latents**:



The total number of steps T is usually set to a large number, such as T=1000

One important thing to know is that these latents are of the same dimensionality as that of the original data .

1.2 Components of the Forward Process

1.2.1 Forward diffusion kernel (FDK),

The transition probability, or **forward diffusion kernel (FDK)**, describes how to get the noisy state x_t from the previous state x_{t-1}

$$q(x_t \mid x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

Breakdown of the Equation:

1. **Distribution Type:** N(…) means the new state xt is sampled from a **Normal (Gaussian) distribution**.
2. **Mean Term ($\sqrt{1-\beta_t}x_{t-1}$):** This is the deterministic part derived from the previous state. The factor $\sqrt{1-\beta_t}$ slightly scales down the previous image x_{t-1}. This scaling ensures that as noise is added, the overall variance of the data doesn't explode, providing consistently scaled inputs.
3. **Variance Term ($\beta_t I$):** This represents the noise added at this specific step. β_t is the scalar from the variance schedule, and I is the identity matrix, meaning the noise is **isotropic Gaussian** (the same amount of noise is applied independently to every dimension/pixel)

In Algebraic terms the above transition can be represented as

$$x_t = \underbrace{\sqrt{1-\beta_t}x_{t-1}}_{\text{Deterministic Signal Decay}} + \underbrace{\sqrt{\beta_t}\epsilon_{t-1}}_{\text{Stochastic Noise Injection}}$$

1.2.2 Variance schedule

The **variance schedule**, denoted as the sequence $\beta_1,\ldots,\beta_T$, is a critical hyperparameter set that governs the rate of information decay throughout the forward diffusion process. It defines the step size of noise injection at each timestep t, determining precisely how much Gaussian noise is added relative to the remaining signal. The schedule must be carefully calibrated to satisfy two competing constraints: the steps must be sufficiently infinitesimal (small βt) to uphold the theoretical requirement that the reverse transitions q(xt−1 | xt) remain approximately Gaussian , yet the cumulative noise over T steps must be large enough to ensure that the final latent state xT effectively converges to a standard isotropic Gaussian N(0,I).

A linearly increasing noise schedule (where β_t grows from β_1=10−4 to β_T=0.02) results in the noise variance being at its maximum magnitude at the "end" of the forward process (t≈T). This structural choice has a profound impact on the generative behavior of the model, specifically enforcing a **"coarse-to-fine"** synthesis hierarchy.

a. The Formation of Global Structure (High t)

Because the variance β_t is largest at the high timesteps (t≈1000), the reverse process begins by removing massive amounts of noise in each step.

- **Large Steps:** The model is forced to make significant, high-variance updates to the latent state during these early generation steps.
- **Impact:** These "large steps" correspond to the formation of **low-frequency, global features**. In this phase, the model is not deciding on textures or eyelashes; it is deciding the fundamental layout, shapes, and pose of the subject.

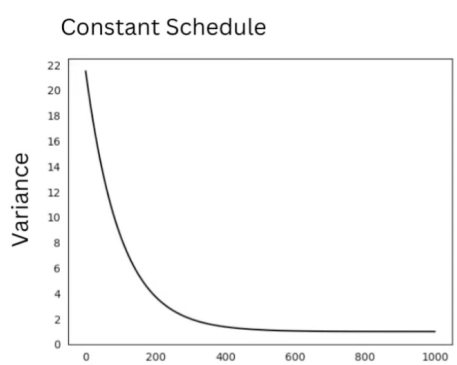
b. Ensuring Prior Convergence

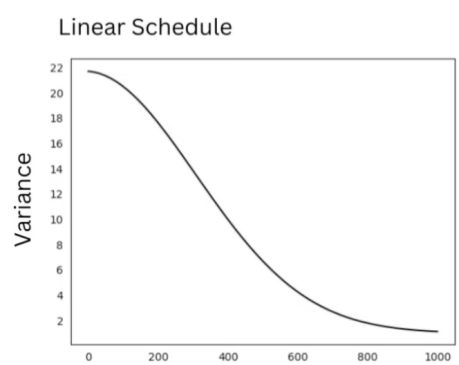
The linear increase ensures that the destruction of information accelerates as time progresses. By ramping up to a larger β_T=0.02, the schedule guarantees that by step T, the signal-to-noise ratio is sufficiently low to render the data indistinguishable from isotropic Gaussian noise. This is critical for the validity of the generative process, which assumes the starting point x_T is pure noise N(0,I).

c. Transition to Fine Details (Low t)

Conversely, because the schedule is linear, the variance βt is smallest at the "end" of the generation process (t→0).

- **Small Steps:** As the reverse process approaches t=0, the noise removal becomes infinitesimal.
- **Impact:** This constrains the model to make only microscopic adjustments, effectively "locking in" the large structure generated earlier and focusing entirely on **imperceptible details** and high-frequency textures.





The above figure clearly showcase the smoother transition of variance in case of a linear schedule compared to a constant schedule

1.2.3 The Reparameterization Trick and Closed-Form Sampling

The **reparameterization trick** and **closed-form sampling** are foundational mechanisms that enable efficient training. Without these, the model would require iteratively computing every single timestep of noise addition (from t=1 to t=T) for every training iteration, which would be computationally intractable.

a. The Gaussian Reparameterization Trick

The forward diffusion process is defined as a Markov chain where a small amount of Gaussian noise is added at each step. Formally, the conditional distribution at step t given t−1 is:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

To sample from this distribution in a way that is differentiable (or simply computationally explicit), we employ the **reparameterization trick**. Instead of treating x_t merely as a sample from a distribution, we express it as a deterministic transformation of the previous state x_{t-1} and an independent noise variable ϵ:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_{t-1}$$

b. Derivation of Closed-Form Sampling

A critical property of the forward process is that we can sample x_t at any arbitrary timestep t directly from the original data x_1, bypassing all intermediate steps $x_1, ..., x_{t-1}$.

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ be the cumulative product of the noise schedule terms. By recursively applying the reparameterization trick, we can derive the direct relationship between x_t and x_0:

1. **Step t:**

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon_{t-1}$$

2. **Step $t-1$:**

$$x_{t-1} = \sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}$$

3. **Substitution:** Substituting the expression for x_{t-1} into the equation for x_t:

$$x_t = \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1-\alpha_t}\epsilon_{t-1}$$

$$x_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + (\sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1})$$

Here, we utilize the property that the sum of two independent Gaussian distributions $\mathcal{N}(0, \sigma_1^2 I)$ and $\mathcal{N}(0, \sigma_2^2 I)$ is itself a Gaussian $\mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)I)$. Thus, the two noise terms merge into a single noise term with a combined variance.

By continuing this recursion down to x_0, we obtain the **closed-form transition equation**:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t)I)$$

c. Impacts and Computational Significance

The rigorous application of the reparameterization trick provides two distinct advantages in this framework:

- **Training Efficiency ($O(1)$ Sampling):** Without this derivation, training the model at an arbitrary timestep t (e.g., t=500) would require simulating the Markov chain sequentially

from t=1 to t=500, an O(T) operation. The reparameterized closed-form equation allows the model to sample xt directly from x0 in constant O(1) time. This enables efficient optimization of the loss function via Stochastic Gradient Descent by randomly sampling t for each batch.

- **Gradient Stability:** While parameters βt are often fixed in DDPM , the explicit separation of signal x0 and noise ϵ stabilizes the training objective. It allows the loss function to be formulated simply as a distance metric between the *true noise* ϵ and the *predicted noise* ϵ_θ, which has been empirically shown to yield superior sample quality.

2. The Reverse Diffusion Process: Creating Order from Chaos

The core innovation of Denoising Diffusion Probabilistic Models (DDPM) lies in their ability to reverse the arrow of time. While the forward process systematically destroys data by adding Gaussian noise until it becomes unrecognizable, the **Reverse Process** aims to reconstruct the original data from pure chaos.
From a statistical viewpoint, the reverse process is a **Markov Chain**,a sequence of events where the future depends only on the present. The goal is to calculate the conditional probability q(xt−1 | xt). In simple terms, the model asks: *"Given the noisy image I have now (xt), what did it look like one step ago (xt−1)?"*
Mathematically, we should be able to calculate this using Bayes' Theorem:

$$q(x_{t-1}|x_t) = \frac{q(x_t|x_{t-1}) \cdot q(x_{t-1})}{q(x_t)}$$

However, this equation is impossible to solve directly. The denominator $q(x_t)$represents the probability of seeing this specific noisy image x_t in the entire database . Calculating this would require integrating over every possible image in the universe to see if it could have diffused into xt.

Since we cannot calculate the exact step, we train a **neural network** p_θ to approximate it. The paper relies on a crucial theorem: i**f the noise steps added during the forward process are small enough, the reverse step is guaranteed to be a Gaussian distribution.**

This simplifies the math immensely. The neural network does not need to predict a complex shape; it only needs to predict two numbers (parameters) to define a Gaussian bell curve:

1. **Mean (μθ):** The "center" of the less-noisy image.

2. **Variance (Σθ):** The uncertainty or "spread" of the guess.

The Simplified Math: To take one step back from xt to xt−1, the model essentially uses this formula:

$$x_{t-1} = \underbrace{\mu_\theta(x_t, t)}_{\text{Predicted Center}} + \underbrace{\sigma_t z}_{\text{Random Noise}}$$

The equation dictates that the image should be shifted toward its predicted center, with the inclusion of a minor, randomized element (z) to prevent it from becoming a completely deterministic blur.

While the probabilistic view talks about predicting the "mean," the practical implementation uses a clever trick. The authors found that instead of training the network to predict the image xt−1 directly, it is much better to train it to **predict the noise** ϵ. The network $\epsilon_\theta(x_t, t)$looks at a noisy image and outputs a noise map. It is essentially a "noise scanner." Once the model predicts the noise, we can mathematically "subtract" it from the current image to estimate the previous step.

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$$

Simplified Explanation:

1. **Take the current image** (x_t).
2. **Subtract the predicted noise** (ϵ_θ), scaled by a specific factor derived from the schedule.
3. **Rescale** the result to maintain the correct variance ($1/\sqrt{\alpha_t}$).
4. **Add random jitter** ($\sigma_t z$) to keep the process generative and diverse.

2.1 The Score Function

In physics, if you have a probability density function p(x) (which describes where data is likely to be found), the **Score** is the gradient (slope) of the log-probability: $\nabla_x log(p(x))$.

- Think of the data distribution as a mountain range. High peaks are "real images," and flat valleys are "noise."

- The **Score** is a vector that points **uphill**. It tells you which direction to move to make the image "more realistic."

2.2 Connection to Diffusion

The paper proves mathematically that **predicting the noise is equivalent to estimating the score**.

$$\epsilon_\theta(x_t, t) \approx -\text{Score}(x_t)$$

When the network predicts the noise, it is actually calculating the gradient of the data density. It is figuring out which direction "uphill" is.

2.3 Langevin Dynamics

In the context of generative modeling, we use a discretized version of the Langevin equation to generate samples from a probability distribution p(x) using only its score function $\nabla_x logp(x)$.

The iterative update rule is defined as:

$$x_{t-1} = x_t + \frac{\delta}{2}\nabla_x \log p(x_t) + \sqrt{\delta} z_t$$

Where:

- xt: The current state of the data (image).
- δ: The step size (analogous to the learning rate).
- $\nabla_x logp(x)$: The **Score Function**. This is the gradient of the log-probability density. It points in the direction where the data is "more likely" (uphill on the density map).
- $z_t \sim N(0, I)$: Standard Gaussian noise.

Interpretation: "To find a realistic image, take the current noisy image, take a small step in the direction that makes it look 'more real' (the score), and add a tiny bit of random noise to ensure you explore the distribution fully and don't just get stuck at the nearest peak."

2.4 Connection to Score Matching

The fundamental breakthrough in modern generative AI is the realization that we do not need to know the probability distribution p(x) itself (which is impossible to calculate for images). We only need its **gradient** (the score).

- **Traditional MCMC:** Methods like Metropolis-Hastings require evaluating the ratio

p(x′)/p(x), which is hard.

- **Langevin Dynamics:** Only requires $\nabla_x logp(x)$.

The Diffusion Link: As established in the DDPM paper, the neural network ϵθ(xt,t) trained to predict noise is essentially learning a scaled version of this score function:

$$\epsilon_\theta(x_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\nabla_x \log p_t(x_t)$$

Therefore, when the diffusion model subtracts the predicted noise, it is mathematically performing a Langevin update step: it is pushing the noisy image up the gradient of the data density.

2.5 Annealed Langevin Dynamics (The "Annealing" Factor)

Standard Langevin dynamics can be slow to mix (converge) if the distribution has isolated modes (e.g., separate islands of high probability). The particle might get stuck on one island and never jump to the others.

Diffusion models solve this by using **Annealed Langevin Dynamics**:

- **Start, t=T:** The noise scale is huge. The distribution is flattened (smoothed). The Langevin process can easily move across the entire space, finding the general region of the data.
- **End, t=0:** The noise scale is tiny. The distribution is sharp. The process refines the local details.

This "coarse-to-fine" annealing ensures that the sampling process doesn't just find *a* valid image, but can reliably explore the diversity of the entire dataset.

3. Variational Inference and the Optimization Objective

3.1 Introduction: The Intractability of Maximum Likelihood

The fundamental objective of any generative model is to maximize the likelihood of the observed data, $p_\theta(x_0)$. Ideally, we would optimize the model parameters θ such that they assign high probability to samples from the true data distribution. However, in the context of

diffusion models, the marginal likelihood $p_\theta(x_0)$ is computationally intractable. Calculating it would require marginalizing over all possible latent trajectories $x_{1:T}$:

$$p_\theta(x_0) = \int p_\theta(x_{0:T}) dx_{1:T}$$

This integration involves summing over high-dimensional latent spaces at every timestep, which is impossible to compute directly. To circumvent this, we employ Variational Inference. Instead of maximizing the exact likelihood, we maximize a rigorous lower bound known as the Variational Lower Bound (VLB) or Evidence Lower Bound (ELBO).

3.2 Deriving the Variational Lower Bound

We begin by expressing the log-likelihood of the data. By introducing the approximate posterior $q(x_{\{1:T\}}|x_0)$,which corresponds to our fixed forward diffusion process,we can multiply and divide the likelihood term by this posterior without changing its value. Applying Jensen's Inequality (which states that $log(E[y]) \geq E[log(y)]$) allows us to push the logarithm inside the expectation, yielding the lower bound:

$$\begin{aligned}
\log p_\theta(x_0) &= \log \int p_\theta(x_{0:T}) dx_{1:T} \\
&= \log \int \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} q(x_{1:T}|x_0) dx_{1:T} \\
&= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right] \\
&\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]
\end{aligned}$$

This resulting expectation is the Variational Lower Bound (L_{VLB}). Maximizing this bound effectively pushes the model's data distribution $p_\theta(x_0)$closer to the true distribution.

3.3 Decomposing the Objective

To make this objective actionable, we must expand the joint probability terms. Recall that both the forward process $q$and the reverse process $p_\theta$are Markovian chains. This allows us to factorize the joint distributions as products of their transition probabilities:

$$\begin{aligned}
p(x_{0:T}) &= p(x_T) \, \Pi p_\theta(x_{t-1}|x_t) \\
q(x_{1:T}|x_0) &= \Pi \, q(x_t|x_{t-1})
\end{aligned}$$

Substituting these factorizations into the VLB equation allows us to separate the terms by timestep. Through algebraic manipulation, the objective can be rewritten as a sum of Kullback-Leibler (KL) divergences:

$$L_{\mathrm{VLB}} = \mathbb{E}_q \left[-\log p(x_T) - \sum_{t=1}^{T} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} \right]$$

$$L_{\mathrm{VLB}} = \underbrace{D_{\mathrm{KL}}(q(x_T|x_0)\|p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(x_{t-1}|x_t, x_0)\|p_\theta(x_{t-1}|x_t))}_{L_{t-1}} \underbrace{-\log p_\theta(x_0|x_1)}_{L_0}$$

This decomposition provides a clear interpretation of the optimization landscape:

1. L_T **(The Prior Match):** This term forces the final forward state $q(x_T|x_0)$ to match the standard Gaussian prior $p(x_T)$. Since the forward process is fixed and has no learnable parameters, this term is constant (approx. 0) and is ignored during training.

2. L_{t-1} **(The Denoising consistency):** This is the crucial term. It measures how well the learned reverse transition $p_\theta(x_{t-1}|x_t)$ approximates the **true forward posterior** $q(x_{t-1}|x_t, x_0)$.
3. L_0 **(The Reconstruction):** This term handles the final decoding step from latent x1 to data x0 ensuring the final output maps correctly to discrete pixel values.

3.4 The Forward Posterior: The "Ground Truth"

To minimize the term L_{t-1}, we essentially need a "teacher" distribution to train our student network p_θ. This teacher is the **forward posterior** $q(x_{t-1}|x_t, x_0)$.
Normally, reversing a diffusion step $q(x_{t-1}|x_t)$ is intractable. However, if we condition on x_0 (the original clean image), the reverse step becomes tractable and closed-form. Using Bayes' rule, we can derive this distribution analytically:

$q(x_{t-1}|x_t, x_0)$= $q(x_t|x_{t-1})$ $q(x_{t-1}|x_0)/q(x_t|x_0)$

Since all terms on the right side are Gaussians (defined by the forward process schedule), the result is also a Gaussian.

Consequently, the model parameterization is adjusted: instead of predicting the mean image, the network $\epsilon_\theta(x_t, t)$is trained to predict the noise ϵ. The loss term L_{t-1} transforms into a

weighted Mean Squared Error between the true noise ϵ and the predicted noise ϵ_θ:

$$L_{t-1} = \mathbb{E}_{x_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(x_t,t)\|^2\right]$$

3.5 The Final Simplified Loss Function (L_{simple})

Plugging the noise-based parameterization back into the KL divergence equation yields a weighted Mean Squared Error (MSE) loss:

$$L_{t-1} = \mathbb{E}_{x_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(x_t,t)\|^2\right]$$

While this is the mathematically correct VLB objective, the authors found that the weighting term in front causes instability and slower convergence. It emphasizes small t (where noise is low) too heavily.
To fix this, they proposed the Simplified Objective L_{simple}, which discards the weighting coefficient entirely:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\right]$$

This simplification essentially re-weights the variational bound to provide a better curriculum for learning. It reduces the penalty for errors at very low noise levels (where the image is already clean) and focuses the model's capacity on high-noise levels (where the structural content is determined).

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1,\ldots,T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$
5: Take gradient descent step on
$\nabla_\theta \left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)\right\|^2$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0},\mathbf{I})$
2: **for** $t = T,\ldots,1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t)\right) + \sigma_t\mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

F. The Neural Architecture: A Time-Conditional U-Net

At the operational core of the Diffusion Models lies a highly specialized neural network designed to reverse the entropy of the forward process. We do not employ a generic feed-forward architecture; instead, they adopted a **U-Net** backbone, a convolutional architecture originally developed for biomedical image segmentation that excels at processing multi-scale visual information. The network is structured as a symmetric encoder-decoder: the encoder (or contracting path) progressively downsamples the image to compress the noisy input into a compact latent representation, effectively isolating global semantic structure. Simultaneously, the decoder (or expanding path) upsamples this representation back to the original pixel resolution. Crucially, the architecture incorporates **residual skip connections** that bridge the encoder and decoder at corresponding resolutions. These connections act as information highways, allowing the model to retain high-frequency spatial details that would otherwise be lost during the aggressive downsampling of the encoding phase, ensuring the final output preserves the sharpness and fidelity of the original data.

A standard U-Net, however, is insufficient for diffusion because the task changes fundamentally over time; removing noise at t=1000 requires a completely different strategy than refining textures at t=1. To address this, the architecture is made **time-conditional** through the injection of sinusoidal position embeddings, a technique adapted from Transformer models. The timestep t is encoded into a high-dimensional vector and added into each residual block of the network. This mechanism effectively modulates the network's internal activation statistics, functioning as a temporal compass that signals the model to prioritize coarse structure generation at high noise levels and fine-grained texture refinement at low noise levels.

To further enhance the model's generative coherence, the convolutional backbone is augmented with **Self-Attention mechanisms**, specifically at the 16×16 feature map resolution. While convolutions are adept at extracting local features, they lack a global receptive field; the attention blocks allow the model to process long-range dependencies across the entire image simultaneously, ensuring global semantic consistency (e.g., structural symmetry) even while operating on local patches. Finally, to ensure training stability across the vast dynamic range of input noise, the standard Batch Normalization is replaced with **Group Normalization**. This architectural choice renders the optimization robust to batch size variations and ensures consistent feature scaling throughout the rigorous training process.

G. Experimental Evaluation and Generative Performance

Empirical evaluations, focused on the CIFAR10 (32x32) and LSUN (256x256) datasets, were conducted to validate the theoretical efficacy of Diffusion Models . The core aim was to test the hypothesis that a DDPM trained with a simplified, non-weighted objective could surpass the inherent constraints of traditional likelihood-based models and achieve perceptual image fidelity comparable to state-of-the-art Generative Adversarial Networks (GANs). The results definitively confirmed this potential. Specifically, on the unconditional CIFAR10 benchmark, the diffusion

model achieved a state-of-the-art Fréchet Inception Distance (FID) of 3.17 and an Inception Score (IS) of 9.46. This performance not only significantly surpassed similar likelihood-based architectures such as PixelCNN and Energy-Based Models but also rivaled the top-tier GANs of the time, thereby successfully bridging the gap between stable probabilistic training and the synthesis of high-fidelity images.

A critical, albeit paradoxical, finding emerged from the analysis of the log-likelihood metrics. While the model excelled at visual generation, its lossless codelengths (Negative Log-Likelihood) were not competitive with other likelihood-based models. The authors attribute this discrepancy to the nature of the diffusion objective, which operates as a form of lossy compression. The majority of the model's capacity is allocated to modeling imperceptible high-frequency details,essentially the noise floor of the image,rather than solely optimizing for bit-rate efficiency. This suggests that diffusion models possess a unique inductive bias: they prioritize perceptual realism and structural coherence over mathematical compression, making them excellent "lossy compressors" but suboptimal "lossless compressors" .

Model	ImageNet	CIFAR
Glow (Kingma & Dhariwal, 2018)	3.81	3.35
Flow++ (Ho et al., 2019)	3.69	3.08
PixelCNN (van den Oord et al., 2016c)	3.57	3.14
SPN (Menick & Kalchbrenner, 2018)	3.52	-
NVAE (Vahdat & Kautz, 2020)	-	2.91
Very Deep VAE (Child, 2020)	3.52	2.87
PixelSNAIL (Chen et al., 2018)	3.52	2.85
Image Transformer (Parmar et al., 2018)	3.48	2.90
Sparse Transformer (Child et al., 2019)	3.44	**2.80**
Routing Transformer (Roy et al., 2020)	**3.43**	-
DDPM (Ho et al., 2020)	3.77	3.70
DDPM (cont flow) (Song et al., 2020b)	-	2.99
Improved DDPM (ours)	**3.53**	**2.94**

Fig :- Comparison of negative log-likelihood of improved DDPM with other likelihood-based generative models

Qualitatively, the samples generated across the LSUN Bedroom and Church datasets demonstrated that the model scales effectively to higher resolutions without suffering from mode collapse, a common failure mode in adversarial training . The generated images exhibited consistent long-range spatial structure, such as the symmetry of cathedral arches or the layout of bedroom furniture, while maintaining realistic fine-grained textures. Furthermore, ablation studies on the training objective revealed a stark contrast in performance: predicting the noise ϵ using the simplified Mean Squared Error loss consistently yielded higher sample quality than predicting the distribution mean μ or using the mathematically rigorous variational bound . This

empirical evidence solidified the "predict-the-noise" paradigm as the optimal strategy for diffusion-based synthesis.

H. Future Directions: The Evolution toward Latent Diffusion

While the Diffusion Models established a new baseline for generative sample quality, its operation in pixel space presents a significant computational bottleneck. As noted in the analysis of the training objective, the model allocates a disproportionate amount of its capacity to modeling imperceptible high-frequency details, essentially "pixel-level noise", rather than the semantic content of the image. This inefficiency makes training high-resolution models (e.g., 1024×1024) prohibitively expensive, as the diffusion process must be simulated for millions of pixels simultaneously. Consequently, the primary trajectory for future research lies in decoupling the "perceptual compression" (learning distinct details) from the "semantic generation" (learning what the image actually is), a paradigm that culminates in the development of **Latent Diffusion Models (LDMs)**, the architecture underpinning systems like Stable Diffusion.

The critical innovation of Latent Diffusion is the shift of the diffusion process from high-dimensional pixel space to a compressed, low-dimensional **latent space**. This is achieved through a two-stage approach. First, a powerful autoencoder (such as a VQ-GAN) is trained to compress images into a latent representation where the high-frequency, imperceptible details are abstracted away, leaving only the essential semantic structure. Second, the diffusion model is trained to generate these *latent vectors* rather than raw pixels. By operating in this compressed space (often 4× to 8× smaller than the original image), the diffusion model requires significantly fewer computational resources and can focus entirely on structural and conceptual modeling rather than pixel-level redundancy. This shift solves the efficiency paradox observed in the original work, allowing for rapid inference on consumer-grade hardware without sacrificing visual fidelity.

Furthermore, the evolution of diffusion models will likely focus on **conditional synthesis**, transforming them from random noise generators into controllable tools for text-to-image creation. While the original DDPM was unconditional, future architectures incorporate **Cross-Attention mechanisms** directly into the U-Net backbone. In this setup, conditioning inputs,such as text descriptions processed by encoders like CLIP,are projected into an embedding space and injected into the diffusion process via attention layers. This allows the diffusion model to attend to specific semantic tokens (e.g., "a photograph of a cat") while denoising, effectively guiding the Langevin dynamics trajectory toward a region of the data manifold that corresponds to the user's prompt. This integration of language understanding with generative diffusion represents the pivotal step toward universal, multimodal generative systems.

I. Real-World Applications and Computational Tools

Diffusion models, which are now fully developed in theory, are quickly shifting from being just an academic curiosity to being used practically across various industries. This boom has given us a ton of new tools that are truly changing how we create digital content. It's much bigger than simply a new way to make art; it's a massive, practical shift in how we handle and shape complex, high-dimensional data in lots of different areas

1. DALL-E 2 (OpenAI)

Representing a seminal moment in multimodal AI, DALL-E 2 demonstrated the profound capacity of diffusion models to bridge the semantic gap between natural language and visual representation. Unlike its autoregressive predecessor, this system utilizes a hierarchical diffusion process, employing a prior to map text embeddings (derived from CLIP) into a corresponding image embedding, which is then decoded into pixel space via a diffusion decoder. This architectural choice allows for "unCLIP" capability,the ability to generate variations of an image by retaining its semantic essence while altering its stochastic details. The tool has found immediate application in creative industries for rapid prototyping and concept art, offering a mechanism to visualize abstract linguistic concepts with photorealistic fidelity, effectively functioning as a "text-to-concept" engine for designers and illustrators.

2. Midjourney

While sharing the underlying diffusion mechanics, Midjourney distinguishes itself through a distinct inductive bias towards artistic stylization and painterly aesthetics rather than strict photorealism. Operating primarily through a discord-based interface, it leverages a proprietary modification of the diffusion process,likely involving extensive fine-tuning on artistic datasets and reinforcement learning from human feedback (RLHF),to optimize for aesthetic appeal and composition. This tool has become instrumental in the architectural and graphic design sectors, where it serves as a "stochastic muse," generating high-fidelity textures, lighting references, and compositional layouts that prioritize visual impact over ground-truth adherence, effectively automating the early stages of the mood-boarding process.

3. Drug Discovery and Protein Design (DiffDock / RFdiffusion)

Moving beyond the visual domain, the stochastic nature of diffusion has found a critical application in the physical sciences, particularly in the modeling of molecular structures. Tools like RFdiffusion treat the protein folding problem as a generative task, where the "noise" consists of random atomic coordinates that are progressively denoised into stable, energetically favorable protein backbones. By learning the distribution of valid protein structures, these models can "hallucinate" novel proteins that do not exist in nature but possess specific binding properties. This application is currently accelerating pharmaceutical research, allowing for the computational design of *de novo* binders and enzymes, effectively reducing the search space for potential drug candidates from years of laboratory work to days of computational inference.

4. AudioLDM and Riffusion

The temporal consistency inherent in diffusion processes has also been successfully adapted for the auditory domain. Systems like AudioLDM utilize latent diffusion to synthesize high-fidelity audio waveforms from text descriptions, effectively treating audio spectrograms as images to be generated. This application is transforming sound design in film and gaming, enabling the procedural generation of ambient soundscapes, foley effects, and background music. By learning the continuous manifold of audio frequencies, these tools allow for the "inpainting" of missing audio segments or the "style transfer" of musical genres, offering a new mode of interaction for audio engineers where sound is sculpted rather than recorded.

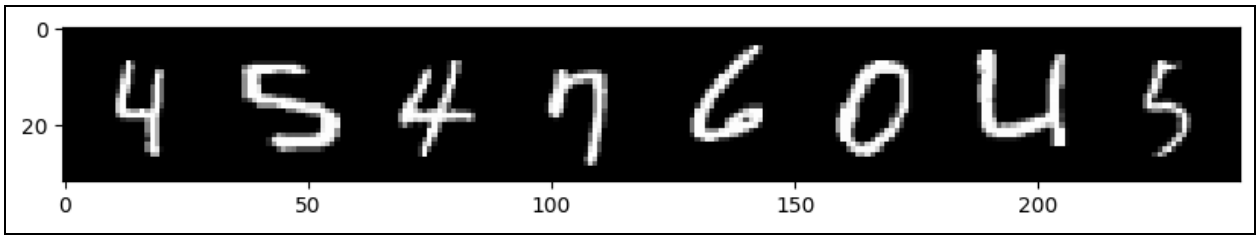
J. Implementation of a DDPM on the MNIST Dataset from scratch

This section describes the complete implementation of a Denoising Diffusion Probabilistic Model (DDPM) trained on the MNIST dataset. The implementation tries to clearly demonstrate the core mechanics of diffusion-based generative modeling. MNIST is chosen due to its low dimensionality and simplicity, which allows clear visualization of the diffusion and denoising processes.

Data Preprocessing

MNIST images are originally stored as pixel intensities in the range 0 to 255. The **`ToTensor()`** transformation converts these values to the range [0, 1]. We then normalize the images using a mean and standard deviation of 0.5, which maps the pixel values to the range [-1, 1]. This normalization is commonly used in diffusion models to center the data around zero, facilitating stable training with Gaussian noise.

A DataLoader was used to sample mini-batches of MNIST images to perform a sanity check. Each batch consists of grayscale images of size 28×28, resulting in tensors of shape (batch_size, 1, 28, 28). The following figure shows a batch of input images after reversing the normalization step for visualization. This confirms that the preprocessing and batching steps are functioning correctly.



In diffusion models, image generation is formulated as a two-stage process: a forward diffusion process that gradually corrupts data by adding noise, and a reverse diffusion process that learns to remove this noise.

Forward Diffusion Process

In the forward diffusion process, noise is added to an image over a fixed number of timesteps T. At each timestep, a small amount of Gaussian noise is injected. After many steps, the image becomes indistinguishable from pure Gaussian noise. This process is fixed and does not involve learning.

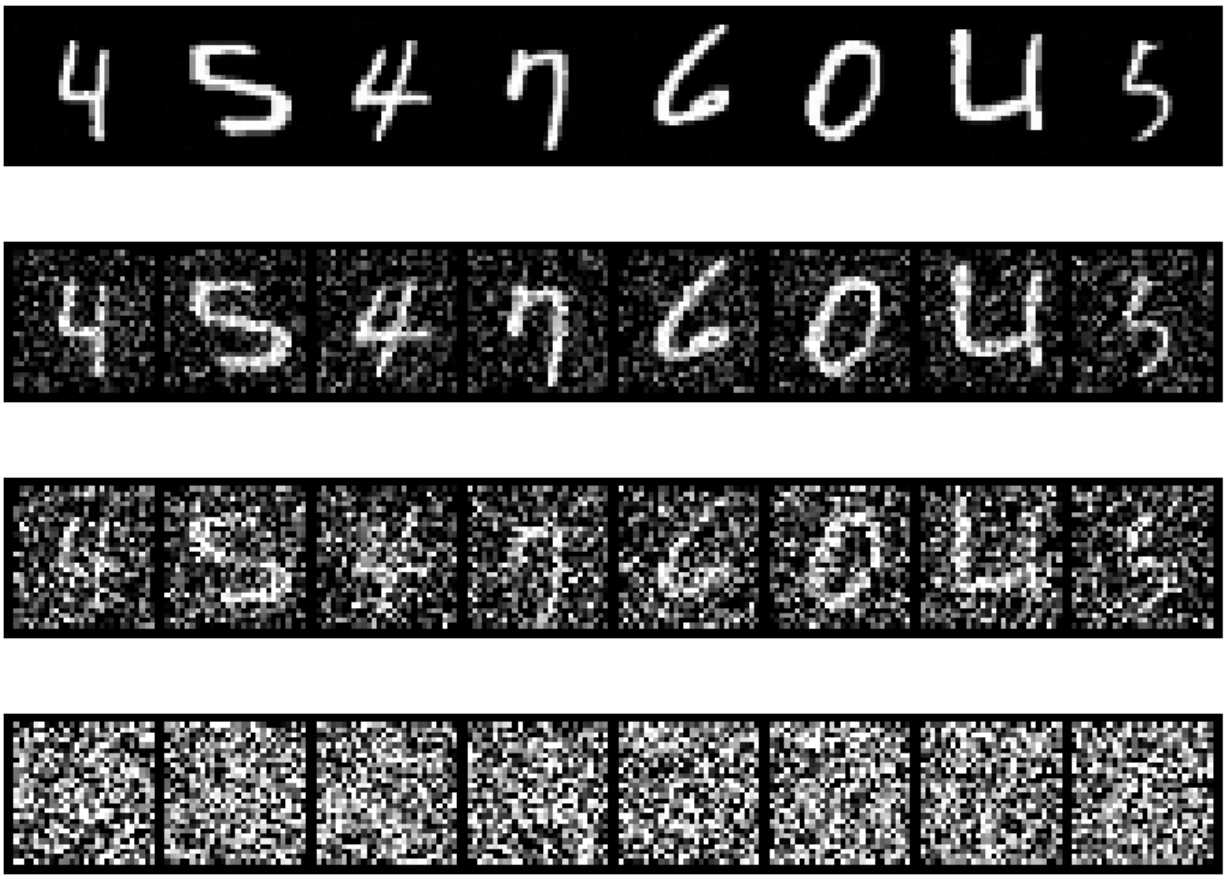
The closed-form equation for forward diffusion is:

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$, where $\varepsilon \sim N(0, I)$

Implementation of the Forward Diffusion Noise Schedule:

```
T = 1000
betas = torch.linspace(1e-4, 0.01, T)
alphas = 1 - betas
alpha_bars = torch.cumprod(alphas, dim=0)
```

Where,
T = number of diffusion steps
β = how much noise is added at each step
$\bar{\alpha}_t$ = represents the proportion of the original image signal preserved at timestep t



The effect of the forward diffusion process is illustrated in the given figures. Starting from clean MNIST images, Gaussian noise is progressively added at increasing timesteps. As the timestep increases, the contribution of the original image diminishes while the noise component becomes dominant. After sufficiently many steps, the images lose all recognizable structure and closely resemble samples drawn from a standard Gaussian distribution.

This gradual noising process enables the learning of a reverse diffusion model that reconstructs meaningful images by iteratively removing noise.

Reverse Diffusion Process and Neural Network Architecture

To model the reverse diffusion process, we employ a convolutional neural network (CNN) based on a simplified U-Net architecture. The role of the network is to predict the Gaussian noise component added to an image at a given diffusion timestep. Once trained, this model allows image generation by reversing the forward diffusion process.

The model **follows an encoder decoder structure**, which allows the network to capture both low-level visual details and higher-level patterns in the image.

The **encoder, or downsampling path**, is composed of three convolutional blocks. Each block applies a 2D convolution followed by group normalization and a SiLU activation function. After the first two blocks, max-pooling is used to reduce the spatial resolution of the feature maps. This gradual reduction in resolution helps the network capture increasingly high-level and abstract representations of the input image.

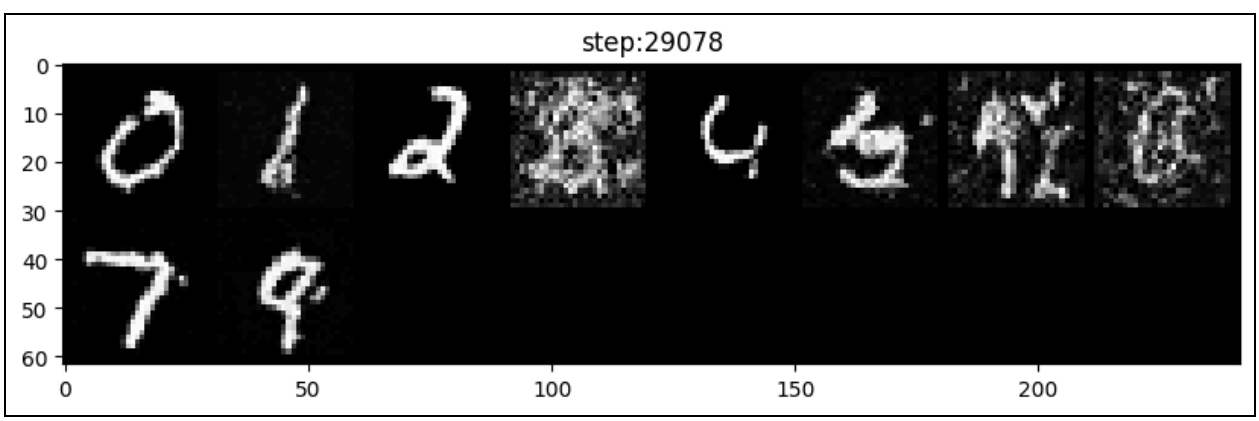
The **decoder, or upsampling path**, follows a symmetric structure and gradually restores the spatial resolution using nearest-neighbor upsampling. Skip connections link corresponding layers of the encoder and decoder, allowing the model to reuse spatial details learned during downsampling. The final convolutional layer produces an output tensor with the same dimensions as the input image, corresponding to the predicted noise.

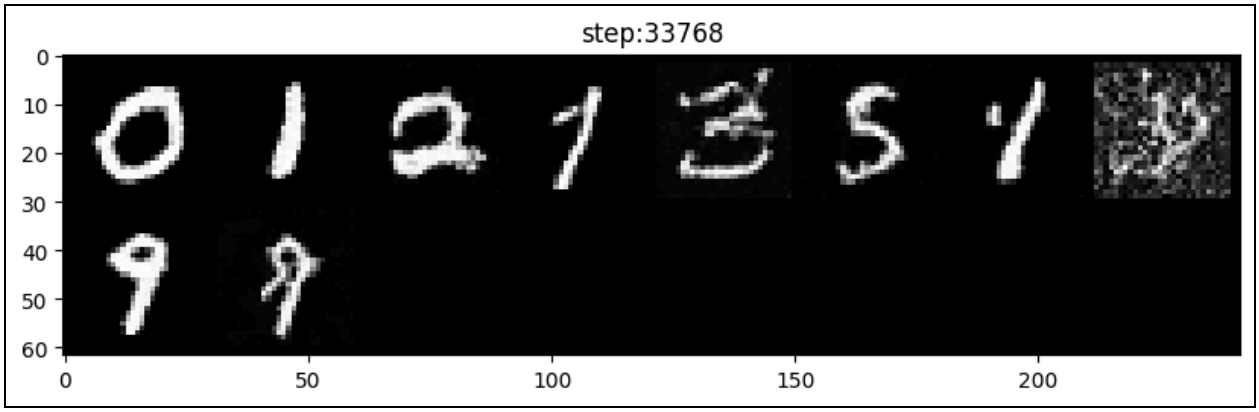
To enable effective denoising across different diffusion steps and digit classes, the model is conditioned on both the diffusion timestep and the class label. This allows the network to adapt its behavior based on the noise level present in the image and to generate class-specific digit samples.

During training, mini-batches of images are sampled from the dataset. For each batch, a diffusion timestep is selected at random, and Gaussian noise is added to the images following the forward diffusion process described earlier.

The forward diffusion process was defined over T=1000 timesteps using a linear noise schedule with β_t linearly increasing from 1×10^{-4} times to 1×10^{-2}. The U-Net model was trained using the AdamW optimizer with a learning rate of 5×10^{-4} and a batch size of 8. Training was performed for 50 epochs by minimizing the mean squared error loss between the predicted noise and the ground-truth noise sampled during forward diffusion.

After training, image generation is performed by reversing the diffusion process. Sampling begins from pure Gaussian noise and proceeds iteratively from the final diffusion timestep to zero. At each step, the model predicts the noise present in the image and uses this prediction to progressively reduce the noise. Through this iterative denoising process, the model transforms random noise into structured images resembling handwritten digits.





Samples generated by the diffusion model at different training steps through the reverse diffusion process outlined in this work.

The complete implementation is available as a Google Colab notebook at:
https://colab.research.google.com/drive/1Eob-xehzvGipa9M2qoHtt-NWPRCRV7Zz?usp=sharing

K . References

- https://learnopencv.com/denoising-diffusion-probabilistic-models/
- https://lilianweng.github.io/posts/2021-07-11-diffusion-models/
- Denoising Diffusion Probabilistic Models by Jonathan Ho , Ajay Jain
- https://github.com/huggingface/diffusers
- https://sander.ai/2023/07/20/perspectives.html
- https://www.assemblyai.com/blog/diffusion-models-for-machine-learning-introduction
- https://andrewkchan.dev/posts/diffusion.html